

Difference Between JPA, Hibernate, and Spring Data JPA

Hands-on Assignment — Spring Core / Spring Data JPA Module

1. Overview

This assignment explains the relationship between three closely related but distinct concepts in Java persistence: the Java Persistence API (JPA) specification, Hibernate (a concrete implementation of that specification), and Spring Data JPA (an abstraction layer built on top of a JPA implementation).

Understanding the distinction between these three is a frequently asked interview topic, since candidates often use the terms interchangeably despite them serving very different roles in the persistence stack.

2. Java Persistence API (JPA)

- JPA is defined by **JSR 338** — a formal specification for persisting, reading, and managing data between Java objects and a relational database.
- JPA itself contains **no concrete implementation**. It only defines interfaces and annotations (such as `@Entity`, `@Id`, `EntityManager`) that describe *what* persistence operations should look like — not *how* they are actually carried out against a database.
- Because JPA is only a specification, it requires a real implementation to do any actual work. **Hibernate is one such implementation** of JPA (other implementations include EclipseLink and OpenJPA).

3. Hibernate

- Hibernate is an **ORM (Object-Relational Mapping) tool** that implements the JPA specification — it is the code that actually executes when a JPA annotation or interface method is used.
- Hibernate introduces its own core objects to manage the database conversation directly: `SessionFactory`, `Session`, and `Transaction`. Using Hibernate directly means the developer is responsible for manually opening a session, starting a transaction, committing or rolling it back, and closing the session — as shown in the code example below.

4. Spring Data JPA

- Spring Data JPA does **not** provide its own JPA implementation — it still relies on Hibernate (or another JPA provider) underneath.
- Its purpose is to **reduce boilerplate code**. It is an additional layer of abstraction sitting on top of a JPA implementation like Hibernate.

- It automatically **manages transactions** declaratively, via the `@Transactional` annotation, removing the need to manually begin, commit, or roll back transactions in application code.

5. Code Comparison: Hibernate vs. Spring Data JPA

The two code samples below implement the exact same operation — creating (saving) an Employee record to the database — first using raw Hibernate, then using Spring Data JPA. Comparing them directly makes the amount of boilerplate Spring Data JPA eliminates immediately visible.

5.1 Raw Hibernate Implementation

```
/* Method to CREATE an employee in the database */
public Integer addEmployee(Employee employee){
    Session session = factory.openSession();
    Transaction tx = null;
    Integer employeeID = null;

    try {
        tx = session.beginTransaction();
        employeeID = (Integer) session.save(employee);
        tx.commit();
    } catch (HibernateException e) {
        if (tx != null) tx.rollback();
        e.printStackTrace();
    } finally {
        session.close();
    }
    return employeeID;
}
```

Line-by-line explanation:

- `Session session = factory.openSession()` — a `Session` represents one single conversation with the database. It must be opened manually before any database work can happen.
- `tx = session.beginTransaction()` — the transaction must be started explicitly, before the save operation.
- `session.save(employee)` — the actual persistence operation. This is the only line that represents the real business intent of the method; everything else is transaction/session plumbing.
- `tx.commit()` — the transaction must be committed manually if the save succeeds.
- `catch (HibernateException e) { tx.rollback(); }` — if anything fails, the transaction must be rolled back manually to avoid leaving the database in an inconsistent state.
- `finally { session.close(); }` — the session must be closed manually in all cases, or the underlying database connection is leaked.

5.2 Spring Data JPA Implementation

EmployeeRepository.java

```
public interface EmployeeRepository extends JpaRepository<Employee, Integer> {

}
```

EmployeeService.java

```
@Autowired
private EmployeeRepository employeeRepository;

@Transactional
public void addEmployee(Employee employee) {
    employeeRepository.save(employee);
}
```

Line-by-line explanation:

- `public interface EmployeeRepository extends JpaRepository<Employee, Integer>` — this single line replaces the entire session-management layer from the Hibernate example. Spring Data JPA generates a working implementation of this interface automatically at runtime, providing `save()`, `findById()`, `findAll()`, and `deleteById()` with zero additional code. The generic parameters specify the entity type (`Employee`) and its primary key type (`Integer`).
- `@Autowired private EmployeeRepository employeeRepository` — Spring's Dependency Injection provides a fully working repository instance automatically; the service never manually constructs it.
- `@Transactional` — this single annotation replaces *all* of the manual `beginTransaction()` / `commit()` / `rollback()` code from the Hibernate example. Spring wraps the annotated method invisibly: a transaction is started before the method runs, committed automatically if the method completes normally, and rolled back automatically if any exception is thrown.
- `employeeRepository.save(employee)` — performs exactly the same underlying operation as `session.save(employee)` in the Hibernate example, but without ever opening or closing a session manually.

6. Summary Comparison Table

Aspect	JPA	Hibernate	Spring Data JPA
What it is	A specification (JSR 338)	An ORM tool; an implementation of JPA	An abstraction layer on top of a JPA provider
Contains code?	No — interfaces and annotations only	Yes — full working implementation	Yes — generates repository implementations
Session/Transaction handling	Not applicable (specification only)	Manual (Session, Transaction)	Automatic, via @Transactional
Boilerplate code	N/A	High (session open/close, tx begin/commit/rollback)	Very low (empty repository interfaces)
Example artifact	@Entity, @Id, EntityManager	SessionFactory, Session, Transaction	JpaRepository, @Transactional

7. Applied Example: orm-learn Project

The same pattern demonstrated above was applied directly in the accompanying `orm-learn` hands-on project, using a `Country` entity instead of `Employee`:

```
// CountryRepository.java - equivalent to EmployeeRepository above
public interface CountryRepository extends JpaRepository<Country, String> {
}

// CountryService.java - equivalent to EmployeeService above
@Autowired
private CountryRepository countryRepository;

@Transactional
public List<Country> getAllCountries() {
    return countryRepository.findAll();
}
```

Calling `countryRepository.findAll()` performs the same kind of work that a raw Hibernate call such as `session.createQuery("FROM Country").list()` would perform — opening a session, executing a query, mapping the result set back into `Country` objects, and managing the transaction — except all of that machinery is generated automatically the moment the repository interface extends `JpaRepository`.

8. Conclusion

JPA, Hibernate, and Spring Data JPA form a layered stack rather than three competing alternatives. JPA defines the contract. Hibernate fulfills that contract with working, if verbose, code. Spring Data JPA sits above Hibernate and removes nearly all of the manual session and transaction handling, letting a developer express persistence intent (such as `save()` or `findAll()`) through a simple interface, while Spring generates and

manages the underlying Hibernate calls automatically.